Computer Shop Management System — Data Structures Documentation

1. Overview

Data structures decide how the program organizes information in memory while it runs. This system uses three of them, each tied to a visible Dashboard feature: a Stack powers Undo Last Extend, Bubble Sort powers Sort by Amount, and Binary Search powers Find Session. All three live in one file, `src/DataStructureManager.java` (under 150 lines, standard library only: `Stack`, `List`, `ArrayList`), operate on the `Session` POJO, and are wired to working UI buttons on the Dashboard — every structure below can be demonstrated live, not just read about. The `ExtendRecord` helper class lives inside the same file because no other class needs to know its shape.

Reading guide: each section follows the same shape — what the structure is, why this system needs it, the exact implementation, its complexity, and a hand-worked trace with real numbers.

2. Stack (LIFO) — Undo Last Extend

2.1 What is a Stack?

A Stack is a pile where you may only touch the top. LIFO means Last In, First Out: the last item placed is the first one removed. It has two operations: `push` (put one item on top) and `pop` (take the top item off).

2.2 Why We Use It

Undo Last Extend must reverse extensions in exact reverse order — the most recent extend is always the one undone first, and each extend must be undone before any earlier one can be touched. That rule is precisely LIFO behavior, so a Stack is the natural fit: each `doExtend()` pushes a record, each Undo pops exactly one.

2.3 Implementation Details

```
public static class ExtendRecord {
    public final int sessionId;
    public final int extraHours;
    public final double extraAmount;
    public ExtendRecord(int sessionId, int extraHours, double extraAmount) {
        this.sessionId = sessionId;
        this.extraHours = extraHours;
        this.extraAmount = extraAmount;
    }
}

private Stack<ExtendRecord> extensionHistory = new Stack<>();

public void pushExtension(int sessionId, int extraHours, double extraAmount) {
    extensionHistory.push(new ExtendRecord(sessionId, extraHours, extraAmount));
}

public ExtendRecord undoLastExtension() {
    if (extensionHistory.isEmpty()) {
        return null;
    }
    return extensionHistory.pop();
}
```

- Class: `DataStructureManager`.
- Field: `Stack<ExtendRecord> extensionHistory`.
- Inner class: `ExtendRecord` stores the session id plus the exact hours and amount that one extend added, so reversal is mathematically exact.
- An empty Stack pops to `null`, which the Dashboard reads as "Nothing to undo."
- UI button: "Undo Last Extend" → `doUndoLastExtend()`.

2.4 Complexity Analysis

Operation	Complexity	Reason
push	O(1)	Adding to the top never scans the pile.
pop	O(1)	Removing the top never scans the pile.

2.5 Flow Example

Session 20 is running. Staff extends twice, then undoes twice:

1. Extend +1 hour (P25.00) → `pushExtension(20, 1, 25.00)`. Stack: `[(20, 1h, P25)]`.
2. Extend +3 hours (P75.00) → `pushExtension(20, 3, 75.00)`. Stack: `[(20, 1h, P25), (20, 3h, P75)]` (top on the right).
3. Click Undo → pops `(20, 3h, P75)`; the database subtracts exactly 3 hours and P75.00. Stack: `[(20, 1h, P25)]`.
4. Click Undo → pops `(20, 1h, P25)`; the database subtracts exactly 1 hour and P25.00. Stack: `[]`.
5. Click Undo → Stack is empty, `pop` is never reached, dialog shows "Nothing to undo."
6. Extend +2 hours → `pushExtension(20, 2, 50.00)`. Stack: `[(20, 2h, P50)]`. The record carries its own values, so this undo will subtract exactly 2 hours even though earlier undos subtracted different amounts. Each pop reverses precisely what its matching push added — never more, never less.

3. Bubble Sort — Sort Sessions by Amount

3.1 What is Bubble Sort?

Bubble Sort walks a list comparing neighbors two at a time and swapping any pair that sits in the wrong order. After each full pass, the most out-of-place element has "bubbled" to its final position, so later passes check a shorter span.

3.2 Why We Use It

It is simple to demonstrate for academic purposes: the nested-loop structure is easy to trace on paper. Session tables are small (a handful of rows per day), so its $O(n^2)$ cost is irrelevant in practice. Our variant sorts descending so the highest-earning sessions appear first.

3.3 Implementation Details

```
public List<Session> sortSessionsByAmount(List<Session> sessions) {
    List<Session> sorted = new ArrayList<>(sessions);
    int n = sorted.size();
```

```
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (sorted.get(j).getAmount() < sorted.get(j + 1).getAmount()) {
                    Session temp = sorted.get(j);
                    sorted.set(j, sorted.get(j + 1));
                    sorted.set(j + 1, temp);
                }
            }
        }
        return sorted;
    }
```

- Input: `List<Session>`, unsorted, any order.
- Output: a NEW list sorted by amount, highest first.
- The original list is never mutated — `new ArrayList<>(sessions)` copies it first, so callers keep their own order.
- UI button: "Sort by Amount" → `doSortByAmount()`, which shows the result in a scrollable dialog.
- Companion: `sortSessionsById()` runs the identical nested-loop shape but compares `getId()` with `>` for ascending order; it exists to prepare input for binary search.

3.4 Complexity Analysis

Case	Complexity	Reason
Best case	O(n)	One pass with no swaps on an already-sorted list.
Average / Worst case	$O(n^2)$	Nested loops compare roughly $n^2/2$ pairs.
Space	O(n)	The defensive copy duplicates the list.

3.5 Step-by-Step Trace

Amounts `[50, 25, 175, 100]`, descending target `[175, 100, 50, 25]`:

- Pass 1 (j = 0..2): 50 vs 25 — in order, no swap → `[50, 25, 175, 100]`; 25 vs 175 — wrong order, swap → `[50, 175, 25, 100]`; 25 vs 100 — wrong order, swap → `[50, 175, 100, 25]`.

After Pass 1, the smallest value (25) has reached its final position at the end of the list. The remaining unsorted portion is [50, 175, 100].

- Pass 2 (j = 0..1): 50 vs 175 — wrong order, swap → `[175, 50, 100, 25]`; 50 vs 100 — wrong order, swap → `[175, 100, 50, 25]`.
- Pass 3 (j = 0): 175 vs 100 — in order, no swap. List is sorted: `[175, 100, 50, 25]`.

The descending order comes from one character: the comparator uses `<` (swap when the left neighbor is SMALLER). Flipping it to `>` produces ascending order — which is exactly what the companion `sortSessionsById` does. Same loops, opposite direction.

4. Binary Search — Find Session by ID

4.1 What is Binary Search?

Binary search finds a value in a SORTED list by divide and conquer: compare the target with the middle element; if it matches, done; if the target is larger, repeat on the right half; if smaller, on the left half. Each step discards half the remaining list. The sorted-input requirement is absolute — on unsorted data it returns wrong answers.

4.2 Why We Use It

It is much faster than checking every row one by one: O(log n) comparisons instead of O(n). The sorted input comes free from `sortSessionsById`, so the pair forms a complete find pipeline. UI button: "Find Session" — staff types a session ID and gets the matching record back in a dialog.

4.3 Implementation Details

```
public Session binarySearchById(List<Session> sessions, int targetId) {
    int left = 0;
    int right = sessions.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int midId = sessions.get(mid).getId();
        if (midId == targetId) {
            return sessions.get(mid);
        } else if (midId < targetId) {
            left = mid + 1;
```

```
            } else {
                right = mid - 1;
            }
        }
        return null;
    }
```

- Precondition: `sessions` sorted by ID ascending (the caller runs `sortSessionsById` first — see `doFindSession`).
- `mid = left + (right - left) / 2` instead of `(left + right) / 2`: adding two large indexes can overflow an `int` and wrap negative; subtracting first keeps every intermediate value in range.
- Returns the matching `Session`, or `null` when no row carries that id (the dialog then reports "not found").

4.4 Complexity Analysis

Case	Complexity	Reason
Best case	O(1)	Target sits at the first middle checked.
Worst case	O(log n)	Each step halves the search span.
Space	O(1)	Iterative — only `left`, `right`, `mid` variables.

4.5 Step-by-Step Trace

The notation below uses the exact formula from the code: mid = left + (right − left) / 2. This form avoids integer overflow. It is NOT (left + right) / 2.

IDs `[1, 2, 3, 4, 5, 6, 7]`, target `6`:

- Iteration 1: left = 0, right = 6, mid = 0 + (6 − 0) / 2 = 0 + 3 = 3 → id 4 < 6, go right: left = 4.
- Iteration 2: left = 4, right = 6, mid = 4 + (6 − 4) / 2 = 4 + 1 = 5 → id 6 == 6, found. Returns the session in 2 steps (a linear scan would need 6).

Miss trace, same list, target `8` (does not exist):

- Iteration 1: left = 0, right = 6, mid = 0 + (6 − 0) / 2 = 3 → id 4 < 8, go right: left = 4.

- Iteration 2: left = 4, right = 6, mid = 4 + (6 − 4) / 2 = 5 → id 6 < 8, go right: left = 6.
- Iteration 3: left = 6, right = 6, mid = 6 + (6 − 6) / 2 = 6 → id 7 < 8, go right: left = 7.
- Loop guard `left <= right` is now 7 <= 6, false — loop ends, returns `null`. The dialog reports "not found." Termination is guaranteed: every iteration either returns or shrinks the span.

5. Summary Table

Structure	Purpose	Time Complexity	Where Used
Stack	Undo Last Extend (LIFO)	push O(1), pop O(1)	Dashboard "Undo Last Extend" button
Bubble Sort	Sort sessions by amount	$O(n^2)$	Dashboard "Sort by Amount" button
Binary Search	Find session by ID	O(log n)	Dashboard "Find Session" button

6. Integration with Database

- All three structures work on in-memory `List<Session>` objects loaded through `SessionDAO.getAllSessions()` — the database is read once per button click, then every comparison and swap happens in memory.
- The Stack lives only in memory for the duration of one Dashboard run: closing the window empties it, and reopening starts fresh. Nothing about undo history is stored in any table.
- `session_log` is the permanent counterpart: every extend, stop, and undo writes an audit row, so the history the Stack forgets on close is still reconstructable from the database afterward.
- All three structures read from and write to the same database through the DAO layer (`SessionDAO`, `PCsDAO`) — no structure touches JDBC directly, and no second data source exists.

UI Button	DAO Reads	Structure Applied	DAO Writes
Undo Last Extend	none (history is in-memory)	Stack pop → exact record	`revertLastExtend` UPDATE
Sort by Amount	`getAllSessions()`	Bubble sort (descending copy)	none (display only)

Find Session	`getAllSessions()`	`sortSessionsById` + binary search	none (display only)

Submitted By:

RONALD ALLAN ZAMORA

ANSARY M. ASIS

JAMES RYAN V. GESTA

AL KENNETH M. UNTALAN

KARL JOROSH C. PACULAN

BHONG S. KILATON